Nullmask

Contracts Audit

Yaroslav Tumanov, Oleksandr Cheliadin

October, 2026

Scoring



9.5/10

Email: info@4irelabs.com

Website: 4irelabs.com

CONFIDENTIAL

This document and the Code Review it references are strictly private, confidential, and personal to its recipients and should not be disclosed, copied, distributed, or reproduced in whole or in part, or passed to any third party without 4ire technologies OÜ's written approval.

1. **Introduction.** Nullmask requested 4irelabs to review its contracts. There are 4 contract source codes in scope, with child base contracts and libraries. The audited code is the verified source of the contracts deployed on Ethereum mainnet (verified on Sourcify / Etherscan; the bytecode compiled from the reviewed sources was confirmed to match the deployed bytecode byte for byte):

    - **Nullmask.sol** - implementation 0x503DE8FAcCee7993ad21d06066C740367c773e0E
    - **NullmaskProxy.sol** - proxy 0xd64EF1417EB047ed0b54736a5a41F01178C391c6
    - **NullmaskSwap.sol** - linked library 0xc80869af3a2f9521030f5b2ef75fe69455d18fdb
    - **AdminUpgradeController.sol** - 0xcD12496c9821075fD807c3488F3E0020715DED27

2. **Warranty.** Audit is provided on an "as is" basis, without warranty of any kind, express or implied. The Auditor does not guarantee that the Code Review will identify all instances of security vulnerabilities or other related issues.

3. **Executive Summary.** Nullmask is a zero-knowledge privacy pool. Users deposit ETH or whitelisted ERC20 tokens; deposits are screened and credited by an off-chain Guard, after which value moves inside the pool as encrypted notes in a Poseidon2 Merkle tree. Shielded transfers,

withdrawals and swaps (through Uniswap V2) are authorized by zero-knowledge proofs verified on-chain. The pool is a UUPS upgradeable contract behind a proxy; upgrade and admin permissions are delegated to an upgrade controller.

The contracts are carefully written: value-moving functions are protected against reentrancy, protocol fees are bounded by hard-coded caps, the ordinary pause never traps user funds, and the inline documentation explains design decisions and references previous audit findings.

The protocol operates under a controlled administrative model. As of October 19, 2026, according to the Nullmask team, administrative control across Ethereum, Base, BNB Smart Chain, and Robinhood Chain has been transferred to a Safe 2-of-3 multisignature wallet (0xb2b1A7b415DBDf38308ba8206B0aED433Ed0706F).

The previous administrator (0x5142...74e0) retains only the token list manager role. The multisignature setup requires multiple approvals for privileged operations, strengthening administrative security while preserving the ability to perform authorized actions without additional delays.

The Guard operates as a separate wallet responsible for approving deposits. Depositors can independently recover unapproved deposits through withdrawPendingDeposit. The security of user funds also depends on the correctness of the Noir circuits.

4. **Critical Bugs and Vulnerabilities.** No Critical bugs and vulnerabilities have been found in the Nullmask contracts. Two Major issues are described in section 7 (items 7.1.10 and 7.1.12). According to the Nullmask team, both findings are not exploitable under the current production configuration.

5. **Remediation Status.** During the audit, on 7 October 2026, the proxy was upgraded to implementation 0x8b5E7C7399A1b4C15D2ae9ee3cffF40a2468797b. Its source is identical to the reviewed Nullmask.sol; the upgrade links a new version of the NullmaskSwap library (0x3240aB784Aac29ad6ff557ccd50764046911D42B, verified on Etherscan). The new version compares the credited amount with amountOutMin, which resolves 7.2.3 and makes the profitable cyclic swap of 7.2.2 revert when a realistic amountOutMin is set; cyclic paths themselves are still accepted. It also restricts shielded swaps to allowlisted relayers, so users can no longer submit fee-less swaps themselves; withdrawals remain permissionless. The new library was compared with the reviewed version but not reviewed in full; the findings in section 7 refer to the deployment listed in section 1.

   On October 19, 2026, according to the Nullmask team, administrative control of the upgrade controller across Ethereum, Base, BNB Smart Chain, and Robinhood Chain was transferred to a Safe 2-of-3 multisignature wallet (0xb2b1A7b415DBDf38308ba8206B0aED433Ed0706F). The previous administrator (0x5142...74e0) retains only the token list manager role.

   The updated administrative setup requires multiple approvals for privileged operations, improving administrative security while maintaining the existing operational flexibility.

   The team also confirmed that the native asset deposit cap is disabled on all four networks and that pull-based deposits are disabled in production. These settings address the concerns raised in findings 7.1.10 and 7.1.11 under the current production configuration.

6. **WhitePaper and specs review and comparison with the code base.** The review used the protocol description published on the official website **(https://nullmask.io/compliance)** together with the inline

NatSpec documentation of the contracts. The deposit flow described there - deposits are held as pending until the Guard approves them, refused deposits are returned to the sender, and a depositor can take back a pending deposit at any time - matches the contract implementation, as do the listed supported assets and operational roles. Several places where the NatSpec does not match the code are listed in section 7 (7.1.8, 7.1.13, 7.1.14, 7.1.15, 7.1.17, 7.1.18).

Recommendation:

keep the NatSpec consistent with the code, in particular in the places listed above.

7. Line-by-line review.

7.1 Nullmask.sol:

7.1.1. **Acknowledged Line 2. Note:** Floating pragma ^0.8.20. The deployed bytecode was compiled with 0.8.28.

Recommendation:

pin the compiler version: pragma solidity 0.8.28;.

7.1.2. **By Design Line 56. Note:** Error InvalidSwapSelector is declared but never used in Nullmask.sol. It is raised by the NullmaskSwap library (NullmaskSwap.sol line 124), which runs via DELEGATECALL; errors of an external library are not part of the caller's ABI, so the declaration here is what lets clients decode the revert against the pool ABI (explained in the NatSpec of NullmaskSwap.sol, lines 58-60). Without a comment at the declaration it looks like dead code and could be removed during a cleanup, which would make swap selector errors undecodable.

Recommendation:

keep the error and add a comment explaining why it is declared.

7.1.3. **Acknowledged Line 70. Note:** Error FeeTooHigh(uint256 value, uint256 max) is used both for basis points (depositBps, withdrawalBps) and for gas units (settleGasOverhead) at lines 853-855. Off-chain tooling cannot tell the unit from the error.

Recommendation:

use a separate error for the settle gas overhead.

7.1.4. **By Design Line 105. Note:** Constant KEY_ROTATION_COOLDOWN and event ReceivingKeyRotated (line 334) are no longer used; their NatSpec states they are kept for ABI compatibility.

Recommendation:

mark them as deprecated in the public interface documentation so integrators do not rely on them.

7.1.5. **Acknowledged Line 374. Low:** The constructor does not validate _virtualChainId and _uniswapV2Router. A zero router, or a VIRTUAL_CHAIN_ID greater than or equal to the BN254 field modulus, produces an implementation where every swap, or every shielded operation, reverts. This only becomes visible on the first user operation after an upgrade.

Recommendation:

require _uniswapV2Router.code.length > 0 and _virtualChainId < FIELD_MODULUS in the constructor.

7.1.6. **Not Applicable Line 376. Low:** VIRTUAL_CHAIN_ID is an immutable set in the constructor of each implementation and is passed as public input [0] of every proof (lines 1191, 1232, 1329). On every upgrade it is supplied again to the constructor of the new implementation, and nothing checks that it equals the current value - neither the constructor nor _authorizeUpgrade. The contract's protection against double spending relies only on the value of the nullifier (noteNullifiers, txNullifiers). If the circuits derive the note nullifier (or the transaction nullifier) from the chain id, an upgrade to an implementation with a different VIRTUAL_CHAIN_ID gives every previously spent note a new nullifier that the contract has never seen, and the same note can be spent again.

The circuit side could not be verified: the Noir circuits were not provided and are out of scope of this audit, so it is unknown whether the chain id is part of the nullifier derivation. The contract side was demonstrated with a test that uses a modelled withdrawal circuit: a user deposits 1 ETH, withdraws it, and after an upgrade that changes VIRTUAL_CHAIN_ID withdraws the same note a second time, taking 1 ETH from other users' funds. With a nullifier that does not depend on the chain id, the second withdrawal is rejected. Changing UNISWAP_V2_ROUTER does not have this effect: the router is a public input of swap proofs only and does not take part in note nullifiers.

Recommendation:

in _authorizeUpgrade, require that the new implementation returns the same VIRTUAL_CHAIN_ID() as the current one, unless a migration is explicitly intended; and confirm with the circuit developers whether the chain id is used in the nullifier derivation.

7.1.7. **Not Applicable Line 400. Note:** initialize only checks that _upgradeController is non-zero, not that the address can answer canUpgrade. Every admin function and upgradeToAndCall call canUpgrade on this address, so with a wrong address all of them revert. Whether this can be recovered depends on what was passed, because setUpgradeController can only be called by the current controller (line 1160):

- an EOA can call setUpgradeController itself and replace the controller (recoverable);
- a contract able to make arbitrary calls (e.g. a multisig) can do the same (recoverable);
- a contract that cannot call setUpgradeController (for example a token or verifier address passed by mistake in the deployment script) leaves the pool permanently without admin functions and upgrades.

Not applicable to the mainnet proxy, which uses a working AdminUpgradeController; the same applies to setUpgradeController.

Recommendation:

before an address becomes the upgrade controller, make sure it can actually act as one: answer authorization requests and later replace itself. After deployment, verify that the controller grants admin rights to the expected address.

7.1.8. **Acknowledged Line 413. Note:** receive only checks that the sender is the Uniswap V2 router, not that a shielded swap is in progress, although the NatSpec states that ETH reaches the pool "only from the Uniswap router, during a shielded swap". Anyone can send ETH to the pool through the router by setting the pool

as the recipient of their own swap or liquidity removal (e.g. swapExactTokensForETH, removeLiquidityETH). Such ETH belongs to no note, cannot be recovered and counts towards the ETH depositCap. No user funds are at risk.

Recommendation:

align the NatSpec with the actual guarantee.

7.1.9. **Acknowledged Line 443. Note:** The parameter noteNullifiers of _spendNotes shadows the state mapping noteNullifiers (State.sol line 56). The current code is correct, but a future edit that reads noteNullifiers[...] inside this function would read the parameter, not the mapping.

Recommendation:

rename the parameter, e.g. nullifiers.

7.1.10. **Mitigated (Configuration) Line 573. Major:** The ETH gas escrow gasEscrow = msg.value - amount is unbounded, and the ERC20 path (line 578) takes msg.value as escrow without checking the ETH deposit cap at all. Because the ETH depositCap is measured on address(this).balance (line 582 - _checkDepositLimits), any user can fill the whole ETH cap with a minimum deposit carrying a large escrow (or with an ERC20 deposit carrying ETH), block all further ETH deposits, and then recover the funds with withdrawPendingDeposit (in full, or minus the capped check fee on the deposited amount when cancelCheckFee is enabled). The minDeposit/maxDeposit limits only apply to amount and do not prevent this. The Guard can free the cap by settling the pending deposit with approveDeposit or rejectDeposit, which returns the unused escrow to the depositor, but the attack can be repeated; each time the attacker only pays the Guard's gas for

the settlement. Not exploitable while depositCap[ETH] == 0 (current mainnet configuration). Confirmed with a unit test and by Echidna/Medusa fuzzing.

Recommendation:

cap the gas escrow (e.g. a per-chain maxGasEscrow), or count amount + gasEscrow against maxDeposit, and check the ETH cap in the ERC20 path as well. Fix before enabling the ETH deposit cap.

7.1.11. **Mitigated (Configuration) Line 667. Low:** A PullIntent deadline has no upper bound, and a depositor cannot cancel a signed requestId on-chain; the only way to revoke is to remove the ERC20 allowance. The gasFee and the revocation key used by pullDeposit are chosen by the Guard and are not part of the signed intent.

Recommendation:

add a maximum intent lifetime and a cancelPullIntent(requestId) function callable by the depositor.

7.1.12. **Not Applicable (Current Configuration) Line 716. Major:** _isAdmin() asks the upgrade controller canUpgrade(msg.sender, address(0)) - a single boolean with no information about which action is being authorized. The same answer gates actions of very different power: setVerifiers (line 967; replacing a verifier with one that accepts any proof drains the pool, so it is equivalent to an upgrade), setWithdrawalsPaused (line 933; the emergency brake), role appointments (setGuard, setRelayer, setWhitelistManager, setTreasury) and routine configuration. A timelock controller cannot distinguish these: it either delays the emergency brake, or lets setVerifiers bypass the delay that

_authorizeUpgrade applies to upgrades. Confirmed with tests: under a timelock-style controller, the admin replaced the verifiers without any delay, after which anyone could withdraw the whole pool balance; under a controller that refuses the address(0) request, the emergency brake could not be used. Not exploitable with the current AdminUpgradeController, under which the admin can upgrade instantly anyway; the issue arises when the controller is replaced with a timelock, as the NatSpec of IUpgradeController anticipates.

Recommendation:

let the controller distinguish the kind of action it authorizes, so that changing the verifiers is subject to the same delay as an upgrade while the emergency pause stays immediately available. Do this before replacing the controller with a timelock.

7.1.13. **Acknowledged Line 721. Note:** The comment lists setUpgradeController among the functions that "must use _requireAdmin". In the code it is gated by msg.sender == upgradeController (line 1160), not by _requireAdmin.

Recommendation:

update the comment.

7.1.14. **Acknowledged Line 918. Note:** The NatSpec of setPaused says "Same access control as setGuard (guard or admin)", but setGuard is admin-only (line 744).

Recommendation:

correct the NatSpec of setPaused.

7.1.15. **Acknowledged Line 933. Note:** The NatSpec of setWithdrawalsPaused describes it as "the only control that can keep user funds inside the pool" during a drain. It does not stop shieldedTransfer (line 1178), which pays its relay fee (line 1209) out of the pool to the allowlisted msg.sender, and the fee is bounded only by the proof. To stop all value leaving the pool, setPaused has to be called as well.

Recommendation:

document that both pauses are needed to stop all outflows, or make setWithdrawalsPaused also stop fee-paying transfers.

7.1.16. **Accepted Risk Line 1270. Low:** _payWithdrawal only rejects the pool itself as recipient. A withdrawal to address(0) burns ETH, and also burns USDT: TetherToken accepts transfers to the zero address and SafeERC20 accepts its empty return value (verified on mainnet with eth_call).

Recommendation:

add if (to == address(0)) revert InvalidRecipient(to);.

7.1.17. **Acknowledged Line 1324. Note:** The comment says "50 public inputs for shielded swap", while the array passed to the verifier has 52 entries (50 struct words plus the chain id and the router, line 1328). The count 50 refers to the struct words only.

Recommendation:

clarify the comment.

7.1.18. **Acknowledged Line 1370. Low:** The NatSpec of setDepositLimits defines the cap as the "largest total balance the pool may hold for this token". The output of a shielded swap increases the pool

balance of the output token without a depositCap check, so this limit can be exceeded by swapping into the token.

Recommendation:

apply the cap to swap outputs, or document that caps only limit deposits.

7.1.19. **Acknowledged Line 1449. Note:** InvalidProof is unreachable with the deployed Honk verifiers: they revert with their own errors and never return false. Monitoring that relies on InvalidProof will not see failed proofs.

7.2. NullmaskSwap.sol:

7.2.1. **Acknowledged Line 2. Note:** Floating pragma ^0.8.20.

Recommendation:

pin to 0.8.28.

7.2.2. **Partially Mitigated Line 128. Low:** The swap output is measured as a balance delta of the output token. For a cyclic path (path[0] == path[last]), the input debit falls inside the measured window, so the function returns out - amountIn instead of out. A loss-making cycle (always the case for A - B - A, which trades through the same Uniswap V2 pair twice) underflows and reverts. A profitable cycle through three different pairs (A - B - C - A) succeeds, but the note is credited only with the profit: the input amount is lost to the user and stays in the pool unaccounted. Not reachable with the current whitelist (ETH and USDT only), as a cycle needs at least two more whitelisted ERC20 tokens.

Recommendation:

reject swap paths that start and end with the same token.

7.2.3. **Resolved Line 128. Low:** The credited amount is never compared with amountOutMin. _checkPath chooses the plain router functions whenever every token on the path is whitelisted as ERC20 or native, and on that branch the router enforces amountOutMin against its own quote, not against what the pool received. For a token labelled ERC20 that actually charges a fee (for example USDT, whose transfer fee can be enabled by its owner), the note can therefore be credited less than the user's floor. The same missing check lets the cyclic swap of 7.2.2 succeed with a realistic floor. Not reachable with the current whitelist (ETH and USDT only), as the plain router functions are only used for token-to-token swaps between whitelisted ERC20 tokens.

Recommendation:

compare the amount actually received with amountOutMin after the swap.

7.3. **NullmaskProxy.sol:**

The mainnet proxy was deployed through the CreateX factory (0xba5Ed099633D3B313e4D5F7bdc1305d3c28ba5Ed, deployCreate2AndInit, transaction 0xc944d7e1...ee99): the proxy was created, the implementation was set and initialize was executed in a single transaction. Item 7.3.1 therefore does not affect the deployed proxy, but applies to any deployment that does not create and initialize the proxy atomically.

7.3.1. **Not Applicable Line 25. Low:** setImplementationAndCall has no caller restriction: whoever calls it first on a fresh proxy chooses the implementation, the verifiers, the Guard and the upgrade controller. Unlike the standard ERC1967Proxy, which is configured

in its constructor, this proxy is configured by a separate call, so if it is created and configured in separate transactions, anyone can take over the canonical Nullmask address in between.

Recommendation:

always create and initialize the proxy in one transaction, as was done on mainnet. Restricting the caller in the proxy itself would change its code and therefore its CREATE2 address, so it is only an option if the same address on other chains is not required.

7.4. AdminUpgradeController.sol:

7.4.1. Acknowledged Line 2. Note: Floating pragma ^0.8.20.

Recommendation:

pin to 0.8.28.

8. Contracts Scoring Legend:

8.1. **9-10.** The highest-scoring contracts that are ready for the mainnet deployment. During the security audit auditors did not find any critical vulnerabilities or bugs that break contracts logic. Contracts gas-optimized.

8.2. **6-8.** The good-scoring results mean that auditors did not find many critical vulnerabilities or bugs that break contracts logic. Also, some parts of the contracts need to be rewritten or implemented (for instance, some parts of functionality missing).

8.3. **1-5.** The low score means that contracts have many critical bugs, logic errors, vulnerabilities, not ready for the mainnet, not gas optimized, etc., and need to be rewritten from scratch.